# Literature Survey: Reinforcement Learning for a Smart Thermostat Energy Saver

CSCE 642 – Reinforcement Learning, Course Project

Jessica Salguero (UIN 937005006) Omar Khan (UIN 437006822)

## 1 Summary of Current Project Plans

We are developing a reinforcement learning controller for a simulated residential smart thermostat. Our main goal is to balance resident comfort with the cost of electricity. Our baseline implementation will apply tabular Q-learning to a simulated single home, built as a custom Gymnasium environment. The project code and environment will be available through GitHub. To evaluate the controller, we will compare the Q-learning agent against other thermostat baselines, including a standard deadband controller. We will also examine how the learned behavior changes with different weather, occupancy, electricity cost, and reward conditions.

### 1.1 Goal and research question

We are building a reinforcement learning (RL) controller for a simulated single-zone residential smart thermostat. The main question is: *can an RL agent learn anticipatory behavior, such as pre-heating or pre-cooling ahead of occupancy and peak electricity prices, that a reactive deadband controller cannot, and how does that behavior change as we shift the balance between energy cost and occupant comfort?*

### 1.2 Problem formulation

**Environment.** We implement a Gymnasium environment around a first-order resistor–capacitor (RC) thermal model,

$$T_{t+1}^{in} = T_t^{in} + \frac{\Delta t}{C}\left[\frac{T_t^{out} - T_t^{in}}{R} + Q_{\mathrm{HVAC}}(a_t)\right] + \varepsilon_t, \tag{1}$$

where $R$ is thermal resistance, $C$ thermal mass, $Q_{\mathrm{HVAC}}(a_t)$ the heating or cooling power of action $a_t$, and $\varepsilon_t$ small random variation.

**State.** Indoor temperature (about 0.5°F bins for the tabular agent, raw float for DQN), outdoor temperature, hour of day (or a coarse time-of-day bucket), occupancy (home/away), day type (weekday/weekend), current HVAC mode, and a time-of-use price tier (off-peak/peak).

**Actions.** Off, low heat, high heat, low cool, high cool (five discrete actions). A continuous HVAC-power action is a stretch goal.

**Reward.** $r_t = -[\alpha E_{\text{cost}} + \beta\,\text{discomfort} + \gamma\,\text{switch}]$, where discomfort is weighted more heavily when the home is occupied and may grow faster the further the temperature leaves the comfort band. The weights $\alpha, \beta, \gamma$ are the main experimental knob.

### 1.3 Methods, baselines, and evaluation

The primary agent is tabular $\epsilon$-greedy Q-learning written from scratch. Baselines are a standard deadband (hysteresis) thermostat, which is the primary baseline, a random-action agent as a sanity floor, and **a scheduled-setback controller as an explicit third baseline**. Controllers are compared on total energy cost, average comfort violation, percentage of occupied time within the comfort range, and number of HVAC mode changes, **plus the additional comfort and anticipation metrics described in Section 1.4**. Behavioral comparisons vary the outdoor-temperature pattern, weekday vs. weekend occupancy, peak vs. off-peak pricing, insulation quality (via $R$ and $C$), and the reward weights. Stretch goals: DQN with continuous state inputs, historical weather data, continuous control (DDPG/PPO), multiple zones, uncertain occupancy, and a long-horizon energy comparison.

### 1.4 Changes since the proposal

Our overall project plan has remained mostly consistent with our original proposal. The changes are set in bold.

- **Scheduled-setback baseline.** Our original implementation plan included a scheduled-setback controller, but it was not in the "Simplest First Results" baseline list. After reviewing related thermostat-control studies (Section 2.3) that compare learned controllers against programmable or static schedules, we **propose making the scheduled-setback controller an explicit third baseline.** This gives us another rule-based comparison beyond the standard deadband controller.
- **Anticipation metrics.** To test the anticipatory-behavior hypothesis directly rather than infer it from aggregate cost, we **propose two additional metrics:** (i) the indoor-temperature deficit at the moment the resident arrives home, and (ii) the fraction of HVAC energy spent in the $k$ timesteps before an occupancy or peak-price transition.
- **Comfort metric.** Following the observation in Sinergym [19] that percentage-of-time-out-of-range can misrepresent controller quality, we **report mean violation magnitude (degree-hours) alongside the percentage-of-time metric.**

## 2 Related Work

We organize the survey around seven questions our project depends on: which RL algorithms we build on (Section 2.1), what is already known about RL for building control (Section 2.2), how comparable thermostat-level systems work (Section 2.3), how reward design and evaluation metrics are handled (Section 2.4), how occupancy can support anticipatory control (Section 2.5), how deep RL has been applied to HVAC (Section 2.6), and what simulators and thermal models are used (Section 2.7). Section 2.8 summarizes how our project differs.

## 2.1 Algorithmic foundations

**Q-learning.** Watkins and Dayan [1] prove that tabular Q-learning, the off-policy temporal-difference update $Q(s,a) \leftarrow Q(s,a) + \eta\,[\,r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$, converges to the optimal action-value function provided every state–action pair is visited infinitely often and the step size $\eta$ decays appropriately. Sutton and Barto [2] give the practical treatment we follow: $\epsilon$-greedy exploration with a decaying $\epsilon$, discussion of reward design, and model-based extensions such as Dyna that interleave real and simulated updates. Two consequences matter for us. First, the guarantee is asymptotic, so coverage of the table is the practical bottleneck. A naive product of our state features (about 50 indoor bins $\times$ 10 outdoor bins $\times$ 24 hours $\times$ 2 occupancy $\times$ 2 day types $\times$ 3 modes $\times$ 2 price tiers) is roughly $2.9 \times 10^5$ states and $1.4 \times 10^6$ state–action pairs. This is an estimate from our proposed bin widths, and in practice many combinations are unreachable because occupancy and price tier are largely functions of (hour, day type). Coarse time-of-day buckets reduce the count by an order of magnitude, which is the trade-off the proposal's "debating 0–23 vs. buckets" note refers to. Second, including the current HVAC mode in the state makes the switching penalty Markovian, since the penalty depends on the previous action.

**Deep Q-networks.** Mnih et al. [3] replace the table with a neural network $Q_\theta(s,a)$ and stabilize training with experience replay (sampling decorrelated minibatches from a buffer) and a periodically frozen target network used to compute the bootstrap target. Their agent learns from raw pixels on Atari; ours would use a small multilayer perceptron on the continuous version of our state, with the same five discrete actions. This is exactly the proposal's DQN stretch goal, and the main comparison is sample efficiency and policy quality against the tabular agent.

**Continuous control.** DDPG [4] extends the DQN ideas to continuous actions with a deterministic actor $\mu_\phi(s)$ trained by the deterministic policy gradient through a critic $Q_\theta(s,a)$, using replay, slowly updated (soft) target networks, and additive exploration noise. PPO [5] is an on-policy method that constrains each policy update with a clipped surrogate objective, trading sample efficiency for stability and simpler tuning. Both are candidates for our continuous HVAC-power stretch goal.

**Environment interface.** Gymnasium [6] standardizes the `reset`/`step` interface, observation and action spaces, and the separation of `terminated` from `truncated` episodes. Our environment follows this API, which also lets us reuse off-the-shelf DQN/DDPG/PPO implementations for the stretch goals.

## 2.2 Surveys of RL for building control

Wang and Hong [7] review RL for building controls and break each study down by algorithm, state, action, reward, and environment. They find that only about 11% of the studies they surveyed involved real buildings, and name three barriers to adoption: training is slow and data-hungry, robustness and security of the controller need improvement, and generalization across buildings is weak (with transfer learning suggested as a remedy). Mason and Grijalva [8] survey RL for autonomous building energy management and trace HVAC applications back to 1996, covering Q-learning, hybrids of RL with model predictive control (MPC), neural fitted Q-iteration, and actor-critic methods such as A3C; they note, for example, the combination of Q-learning with Bayesian occupancy prediction by Barrett and Linder [10]. Nagy et al. [9] give a broader "ten questions" treatment aimed at buildings acting as flexible grid assets that must shift demand while maintaining comfort.

*Relation to our work.* These surveys supply the taxonomy we use to describe our design (state, action,

reward, environment) and justify why we evaluate in simulation. We do not attempt deployment, so the real-building barriers they identify are out of scope, but the generalization barrier motivates our experiments across insulation levels and weather patterns. The surveys also show that discrete tabular agents are the early, simple end of the literature, which is the niche where a clean, fully inspectable study of anticipatory behavior is still feasible.

### 2.3 Thermostat-level and residential control

**Barrett and Linder (2015)** [10] are the closest prior work to our tabular agent. Motivated by commercial learning thermostats whose algorithms are undisclosed, they propose an architecture with two parts: a Bayesian learner that predicts room occupancy over time, and a Q-learning agent that learns the thermostat control policy, enabled by a purpose-designed state–action formalism. They report cost savings of about 10% over a programmable thermostat while keeping occupant comfort high. *Similarities:* Q-learning, a reward that trades off comfort against energy cost, and a thermostat-scale problem. *Differences:* occupancy is *predicted* in their pipeline, whereas we initially give the controller the true occupancy; their comparison is against a programmable thermostat, ours against a deadband controller and a scheduled-setback controller; and we add time-of-use pricing and a systematic sweep of reward weights. Their pipeline is effectively what our "uncertain occupancy" stretch goal would reproduce.

**PreHeat (Scott et al., 2011)** [11] is a non-RL system that senses occupancy, uses historical occupancy to estimate the probability of future occupancy, and uses that estimate to set the heating schedule automatically. It was deployed in five homes (three in the US, two in the UK) for about 61 days per house on average, alternating days between PreHeat and a static schedule and measuring gas use and "MissTime," the time the house was occupied but not warm. In UK homes PreHeat saved gas and reduced MissTime; in US homes it cut MissTime by a factor of 6–12 at similar gas consumption. *Relevance:* PreHeat is the strongest empirical evidence that anticipatory heating pays off in real homes, and MissTime is essentially the complement of our "percentage of occupied time in the comfort range." *Difference:* the anticipation is hand-engineered (prediction followed by a fixed rule), whereas we ask whether an RL agent *discovers* the same behavior purely from reward, and whether the learned lead time depends on the thermal time constant $RC$.

**Ruelens et al. (2017)** [12] apply batch RL, specifically fitted Q-iteration (FQI), to demand response with thermostatically controlled loads including a residential heat-pump thermostat. They extend FQI to settings where a forecast of exogenous data is available, add a policy-adjustment step that injects expert knowledge, and also construct open-loop day-ahead schedules for market participation. Their experiments show batch RL is a viable alternative to model-based controllers without a system-identification step. *Similarities:* price-responsive control of a thermal load with a learned $Q$-function. *Differences:* FQI learns from a fixed batch of transitions and regresses $Q$ with function approximation, while we learn online with a table; they feed forecasts into the state, and our hour-of-day and price-tier features play the role of a known exogenous forecast in a much simpler form.

### 2.4 Reward design and evaluation metrics

In the related literature, one important design choice is how each controller balances energy consumption against occupant comfort. If a controller focuses on only one of these, the result can be a system that rarely turns on the HVAC because it prioritizes energy cost, or one that runs

at high energy cost because it prioritizes comfort. Because of this balance, many of these studies include penalties for both energy use and occupant discomfort.

Barrett and Linder (2015) [10] use a reward that accounts for energy cost and occupant comfort directly in their Q-learning thermostat controller. *Similarity:* our reward function also requires the agent to balance the cost of electricity against keeping the home within a comfortable temperature range. *Difference:* our reward also includes a switching penalty, which discourages the agent from changing between heating, cooling, and off more often than necessary.

Li et al. (2020) [14] take a different approach: they minimize cooling cost in a data center while keeping temperatures within a required range. *Relevance:* although their application differs from a residential thermostat, their work shows another way of preventing a controller from over-prioritizing low energy cost at the expense of acceptable temperatures.

Sinergym [19] also combines energy use and temperature comfort in its reward functions, and its occupancy-based reward changes the importance of discomfort depending on whether the building is occupied. *Similarity:* this matches our plan to penalize temperature violations more heavily when residents are home. Sinergym also points out that reporting only the percentage of time outside the comfort range may not fully describe controller performance, since it does not show how large the violations are.

*Relation to our work.* Based on these studies, we evaluate our controller on more than its total RL reward. We compare total energy cost, percentage of occupied time within the comfort range, the magnitude of temperature violations, and the number of HVAC mode changes. We also vary the weights on energy cost, discomfort, and HVAC switching to see how these choices change the learned policy.

### 2.5 Occupancy and anticipatory control

Occupancy is central to our project. One of our main research questions is whether the RL agent can learn to heat or cool the home before a resident returns, so that the controller prepares for someone coming home instead of only reacting after the temperature is already outside the comfort range.

Barrett and Linder (2015) [10] combine Q-learning with a Bayesian model that predicts future occupancy of a room, and use the prediction as part of the thermostat control process. *Difference:* our initial implementation is simpler because we provide the current occupancy directly as part of the state rather than using a separate model to predict future occupancy.

PreHeat (Scott et al., 2011) [11] also uses previous occupancy patterns to estimate when residents are likely to be home, but does not use RL; the predicted occupancy is used to create a heating schedule automatically. *Relevance:* PreHeat intentionally heats the home before the resident returns, which is closely related to our goal. In our project we ask whether the reward function and repeated interaction with the simulated environment can lead an RL agent to anticipate when its behavior should change.

Ruelens et al. (2017) [12] give their batch RL approach forecasts of exogenous data. *Difference:* our initial state does not include forecasts. Instead, the agent receives the time of day, day type, current occupancy, and electricity price tier, and these patterns may still let it learn when changes in occupancy or prices are likely to happen.

*Relation to our work.* One limitation of our initial implementation is that the agent only knows the

current occupancy state. If a resident returned home at completely random times, the agent would not have enough information to know when to start heating or cooling in advance. For our initial experiments the occupancy schedule will therefore follow regular patterns that the agent can learn from the time of day and day type. In the future we could add uncertain occupancy or a separate occupancy-prediction model as an extension.

### 2.6 Deep RL for HVAC control

**Wei et al. (2017)** [13] formulate building HVAC control as a Markov decision process and train a deep Q-learning controller in an EnergyPlus-based simulation, controlling the supply airflow rate for building zones; the data-driven approach avoids a detailed physical model at control time. This is the model for our DQN stretch goal. *Differences:* their setting is a multi-zone commercial-scale building simulated at high fidelity, whereas ours is a single-zone residential RC model with occupancy and price features.

**Li et al. (2020)** [14] cast data-center cooling as energy-cost minimization under temperature constraints and propose an end-to-end cooling control algorithm built on an off-policy, offline variant of DDPG. An evaluation network predicts an energy cost penalized by the cooling status of the room, and a policy network outputs control settings given the current load and weather. On EnergyPlus they report about 11% cooling-cost saving relative to a manually configured baseline, and about 15% on a real trace from Singapore's National Supercomputing Centre under a 26.6°C inlet-temperature threshold. *Relevance:* a direct template for our continuous-control extension and for treating comfort as a constraint-like penalty. *Differences:* data-center load is driven by servers, not occupancy, and there is no price signal.

**Gnu-RL (Chen et al., 2019)** [15] targets deployability. The agent's policy is a differentiable MPC layer that encodes planning and linear system dynamics, is pre-trained by imitation learning on historical data from the existing controller (so it behaves reasonably from the start, hence "precocial"), and then improves online with a policy-gradient algorithm. It was tested in EnergyPlus and deployed on the airflow of a real conference room for three weeks. *Relevance:* Gnu-RL shows the benefit of building physics into the policy for sample efficiency and interpretability. Our RC model in Eq. (1) is a linear first-order dynamics model of the kind such a policy would fit, which makes a model-based variant (or Dyna-style planning in the tabular agent) a natural future comparison. *Difference:* our agent is model-free and learns from scratch in simulation, with no pre-training from an incumbent controller.

**Xu et al. (2020)** [16] study transfer learning for HVAC control, proposing a DQN-based method to reuse a trained controller across different building configurations and weather conditions to cut retraining cost. This speaks directly to the generalization barrier noted in [7]. *Relevance:* our planned experiments with better- and worse-insulated homes (different $R$ and $C$) probe the same question at small scale: how brittle is a policy trained on one thermal configuration when the house changes? We do not implement transfer learning.

### 2.7 Simulation platforms and thermal models

**Grey-box RC models.** Bacher and Madsen [17] propose a procedure for choosing a model of a building's heat dynamics from a family of lumped-parameter (grey-box) models of increasing complexity, comparing them with likelihood-ratio tests. These models are written as stochastic differential equations whose parameters (thermal resistances and capacities) have direct physical

meaning. Our single-node model, Eq. (1), is the simplest member of this family: it has time constant $\tau = RC$, relaxes toward $T^{out} + R\,Q_{\mathrm{HVAC}}$ under constant power, and its additive noise $\varepsilon_t$ plays the role of the diffusion term in the stochastic formulation. Because we discretize with forward Euler, we need $\Delta t/(RC) \ll 1$ for the discrete model to track the continuous one, a constraint to check when choosing $\Delta t$. The steady state $T^{out} + RQ$ also tells us whether "low heating" can physically reach the comfort band in cold weather, which constrains how we should set $Q$ for each action. Unlike [17], we do not identify parameters from data; we set them by hand to represent houses of different quality, which is a limitation we will state in the report.

**EnergyPlus-based testbeds.** Moriyama et al. [18] release an open-source wrapper that connects EnergyPlus to RL frameworks and use it to train deep RL on a two-zone data-center model, reporting a 22% improvement over the built-in model-based controller. Sinergym [19] is a more recent Gymnasium-compatible testbed that connects to EnergyPlus through its Python API, supports discrete and continuous actions, interchangeable weather files (with Ornstein–Uhlenbeck noise to vary weather between episodes), and custom reward functions. Its default reward is a weighted combination of energy and comfort violation, and a "schedule" reward varies the discomfort weight with occupancy, which closely matches our occupancy-weighted discomfort term. In its example study, deep RL controllers (PPO, TD3, SAC) saved roughly 15–18% energy over reactive controllers in several months, and PPO was the most stable. The authors also observe that percentage of time outside the comfort range can be a misleading metric compared with the average violation magnitude. *Relation to our work:* these platforms offer high fidelity at a high simulation cost, and their authors note that building simulators are expensive to run and awkward to wrap in an RL loop. Our lightweight RC environment gives up fidelity for millions of cheap timesteps, full control over occupancy and price, and complete reproducibility, which suits a controlled study of how behavior changes with reward weights. Sinergym's reactive rule-based baselines are the analogue of our deadband controller.

**Comfort range.** We define the resident's comfort band from standard thermal-comfort guidance [20]; the band itself is a tunable parameter of the reward and evaluation rather than a modeling claim.

### 2.8 Synthesis: how our project differs

Table 1 summarizes how our project compares with the most closely related work. Overall, these studies show that RL can be useful for HVAC control, and that occupancy information can support heating or cooling before a resident returns. They also show the importance of balancing energy use with occupant comfort in controller design [10, 11, 14, 19]. Other studies show that open challenges remain in how efficiently the agent learns, how reliable the controller is, and whether a trained controller still works when applied to different buildings or environments [7, 15, 16].

Our project focuses on a simpler and more controlled version of this problem. Instead of building a fully robust HVAC controller, we use a small residential simulation and tabular Q-learning so that we can concentrate on understanding what the agent can learn:

1. We use a reproducible, low-cost residential environment that includes occupancy, time-of-use electricity pricing, and a switching penalty.

2. We examine whether anticipatory behavior develops in the tabular Q-learning agent, and how it changes when the energy-cost and comfort weights $(\alpha, \beta, \gamma)$ or the thermal conditions change.

3. We use an inspectable Q-table so that the learned policy can be examined by time of day, occupancy, and electricity price. If time allows, this can also be compared against a DQN as a stretch goal.

Our goal is not to improve on the more advanced methods described above or to make claims about real-world deployment. Instead, we want to use a simple environment to better understand how an RL agent responds to the different parts of the thermostat-control problem.

| **Work** | **Method** | **Setting** | **Key difference from ours** |
|---|---|---|---|
| Barrett & Linder [10] | Q-learning + Bayesian occupancy prediction | Thermostat, simulated | Occupancy predicted, not given; no price tiers; programmable-thermostat baseline |
| PreHeat [11] | Occupancy prediction + schedule (no RL) | Five real homes | Hand-built anticipation; we test whether RL learns it |
| Ruelens et al. [12] | Fitted Q-iteration (batch) | Heat pump, water heater | Batch regression with forecasts vs. our online table |
| Wei et al. [13] | DQN | EnergyPlus building | Multi-zone, high fidelity; no occupancy/price focus |
| Li et al. [14] | Offline DDPG | Data-center cooling | Server load, no occupancy or price |
| Gnu-RL [15] | Differentiable MPC + policy gradient | EnergyPlus; real room | Model-based, imitation pre-training; ours model-free from scratch |
| Xu et al. [16] | DQN transfer learning | Varied buildings/weather | We only test robustness to $R, C$ changes |
| Sinergym [19] | PPO/TD3/SAC on EnergyPlus | Data-center example | High-fidelity platform; ours is a lightweight RC model |

Table 1: Comparison of the most closely related systems with the proposed project.

# References


[1] C. J. C. H. Watkins and P. Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

[2] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[4] T. P. Lillicrap, J. J. Hunt, A. Pritzel, et al. Continuous control with deep reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. arXiv:1509.02971.

[5] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[6] A. Kwiatkowski, M. Towers, J. Terry, et al. Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032, 2024.

[7] Z. Wang and T. Hong. Reinforcement learning for building controls: The opportunities and challenges. *Applied Energy*, 269:115036, July 2020. doi:10.1016/j.apenergy.2020.115036.

[8] K. Mason and S. Grijalva. A review of reinforcement learning for autonomous building energy management. *Computers & Electrical Engineering*, 78:300–312, 2019. doi:10.1016/j.compeleceng.2019.07.019.

[9] Z. Nagy, G. Henze, S. Dey, et al. Ten questions concerning reinforcement learning for building energy management. *Building and Environment*, 241:110435, 2023. doi:10.1016/j.buildenv.2023.110435.

[10] E. Barrett and S. Linder. Autonomous HVAC control, a reinforcement learning approach. In *Machine Learning and Knowledge Discovery in Databases: ECML PKDD 2015, Part III*, Lecture Notes in Computer Science, Springer, 2015. doi:10.1007/978-3-319-23461-8_1.

[11] J. Scott, A. J. B. Brush, J. Krumm, B. Meyers, M. Hazas, S. Hodges, and N. Villar. PreHeat: Controlling home heating using occupancy prediction. In *Proceedings of the 13th International Conference on Ubiquitous Computing (UbiComp '11)*, pp. 281–290. ACM, 2011. doi:10.1145/2030112.2030151.

[12] F. Ruelens, B. J. Claessens, S. Vandael, B. De Schutter, R. Babuška, and R. Belmans. Residential demand response of thermostatically controlled loads using batch reinforcement learning. *IEEE Transactions on Smart Grid*, 8(5):2149–2159, Sept. 2017. doi:10.1109/TSG.2016.2517211.

[13] T. Wei, Y. Wang, and Q. Zhu. Deep reinforcement learning for building HVAC control. In *Proceedings of the 54th Annual Design Automation Conference (DAC '17)*, pp. 1–6. ACM, 2017. doi:10.1145/3061639.3062224.

[14] Y. Li, Y. Wen, D. Tao, and K. Guan. Transforming cooling optimization for green data center via deep reinforcement learning. *IEEE Transactions on Cybernetics*, 50(5):2002–2013, 2020. doi:10.1109/TCYB.2019.2927410.

[15] B. Chen, Z. Cai, and M. Bergés. Gnu-RL: A precocial reinforcement learning solution for building HVAC control using a differentiable MPC policy. In *Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '19)*, 2019. Extended version: *Frontiers in Built Environment*, 2020, doi:10.3389/fbuil.2020.562239.

[16] S. Xu, Y. Wang, Y. Wang, Z. O'Neill, and Q. Zhu. One for many: Transfer learning for building HVAC control. In *Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys '20)*, 2020. arXiv:2008.03625.

[17] P. Bacher and H. Madsen. Identifying suitable models for the heat dynamics of buildings. *Energy and Buildings*, 43(7):1511–1522, 2011. doi:10.1016/j.enbuild.2011.02.005.

[18] T. Moriyama, G. De Magistris, M. Tatsubori, T.-H. Pham, A. Munawar, and R. Tachibana. Reinforcement learning testbed for power-consumption optimization. In *Methods and Applications for Modeling and Simulation of Complex Systems*, Communications in Computer and Information Science, pp. 45–59. Springer, Singapore, 2018. doi:10.1007/978-981-13-2853-4_4.

[19] A. Campoy-Nieves, A. Manjavacas, J. Jiménez-Raboso, M. Molina-Solana, and J. Gómez-Romero. Sinergym – A virtual testbed for building energy optimization with reinforcement learning. arXiv:2412.08293, 2024.

[20] ASHRAE. *ANSI/ASHRAE Standard 55-2020: Thermal Environmental Conditions for Human Occupancy*. 2020.